A Project Report for Compiler Construction

Birla Institute Of Technology and Science Pilani
Hyderabad Campus

Compiler Construction
(CS F363)

Under the supervision of
Prof. Jabez Christopher & Prof. Aruna Malapati

Submitted By: **Group 15**

Sr No.	Name	ID Number
1	VENU SAI YELESVARAPU	2023A7PS0149H
2	ROHAN REDDY DEVARAPALLI	2023A7PS0138H
3	ALBERT SEBASTAIN	2023A7PS0118H
4	AKSHITH VUPPALA	2023A7PS0032H
5	ADITHYA NAMA	2023A7PS0171H

Contents

1 Introduction **2**

2 Context Free Grammar **3**
- **2.1 Start Symbol** . . . 3
- **2.2 Classes** . . . 3
- **2.3 Expressions** . . . 3
- **2.4 Let Expression** . . . 3
- **2.5 Assignment** . . . 3
- **2.6 While Loop** . . . 3
- **2.7 If Statement** . . . 4
- **2.8 Block** . . . 4
- **2.9 Arithmetic Expressions (With Precedence)** . . . 4
- **2.10 Boolean Expressions** . . . 4
- **2.11 Function Call** . . . 4

3 Regular Expression for Tokens **5**
- **3.1 Keywords** . . . 5
- **3.2 Operators and Delimiters** . . . 5
- **3.3 Identifiers and Types** . . . 5
- **3.4 Literals** . . . 6
- **3.5 Whitespace and Comments** . . . 6

4 Setup Instructions **7**

5 Lexical Errors **8**

1 Introduction

COOL (Classroom Object-Oriented Language) contains a number of programming language features common in other object oriented languages (such as Java).

- **Pure OOP:** Strictly object-oriented and requires single inheritance and encapsulation.
- **Static Typing:** Uses type checking at compile time to make sure types are safe.
- **Expression-Based:** Most language constructs are expressions that return a value.
- **Minimalist Syntax:** On purpose, it leaves out complicated syntactic sugar in favor of simple logic.
- **Core Classes:** Uses a simple set of built-in classes that are the building blocks of the language (like Object, Int, and String).

2 Context Free Grammar

2.1 Start Symbol

$$\text{Program} \rightarrow \text{ClassList}$$

2.2 Classes

$$\begin{aligned}
\text{ClassList} &\rightarrow \text{Class ClassList} \\
&\mid \text{Class}
\end{aligned}$$

$$\text{Class} \rightarrow \texttt{class}\ \text{TYPE}\ \{\text{FeatureList}\}\ ;$$

$$\begin{aligned}
\text{FeatureList} &\rightarrow \text{Feature FeatureList} \\
&\mid \varepsilon
\end{aligned}$$

$$\text{Feature} \rightarrow \text{ID}\ (\ )\ :\ \text{TYPE}\ \{\text{Expr}\}\ ;$$

2.3 Expressions

$$\begin{aligned}
\text{Expr} \rightarrow\ & \text{LetExpr} \\
\mid\ & \text{AssignExpr} \\
\mid\ & \text{WhileExpr} \\
\mid\ & \text{IfExpr} \\
\mid\ & \text{Block} \\
\mid\ & \text{ArithExpr} \\
\mid\ & \text{BoolExpr} \\
\mid\ & \text{ID} \\
\mid\ & \text{INT} \\
\mid\ & \text{BOOL} \\
\mid\ & \text{STRING} \\
\mid\ & \text{FunctionCall}
\end{aligned}$$

2.4 Let Expression

$$\text{LetExpr} \rightarrow \texttt{let}\ \text{LetDeclList}\ \texttt{in}\ \text{Expr}$$

$$\begin{aligned}
\text{LetDeclList} &\rightarrow \text{LetDecl}\ ,\ \text{LetDeclList} \\
&\mid \text{LetDecl}
\end{aligned}$$

$$\begin{aligned}
\text{LetDecl} &\rightarrow \text{ID} : \text{TYPE} \\
&\mid \text{ID} : \text{TYPE} \leftarrow \text{Expr}
\end{aligned}$$

2.5 Assignment

$$\text{AssignExpr} \rightarrow \text{ID} \leftarrow \text{Expr}$$

2.6 While Loop

$$\text{WhileExpr} \rightarrow \texttt{while}\ \text{Expr}\ \texttt{loop}\ \text{Expr}\ \texttt{pool}$$

2.7 If Statement

$$\text{IfExpr} \rightarrow \texttt{if}\ \text{Expr}\ \texttt{then}\ \text{Expr}\ \texttt{else}\ \text{Expr}\ \texttt{fi}$$

2.8 Block

$$\text{Block} \rightarrow \{\text{ExprList}\}$$

$$\begin{aligned}\text{ExprList} \rightarrow\ & \text{Expr}\ ; \\ |\ & \text{Expr}\ ;\ \text{ExprList}\end{aligned}$$

2.9 Arithmetic Expressions (With Precedence)

$$\begin{aligned}\text{ArithExpr} \rightarrow\ & \text{ArithExpr} + \text{Term} \\ |\ & \text{ArithExpr} - \text{Term} \\ |\ & \text{Term}\end{aligned}$$

$$\begin{aligned}\text{Term} \rightarrow\ & \text{Term} * \text{Factor} \\ |\ & \text{Term}/\text{Factor} \\ |\ & \text{Factor}\end{aligned}$$

$$\begin{aligned}\text{Factor} \rightarrow\ & \text{ID} \\ |\ & \text{INT} \\ |\ & (\text{Expr})\end{aligned}$$

2.10 Boolean Expressions

$$\begin{aligned}\text{BoolExpr} \rightarrow\ & \text{Expr} < \text{Expr} \\ |\ & \text{Expr}\ \texttt{and}\ \text{Expr} \\ |\ & \texttt{not}\ \text{Expr} \\ |\ & \text{BOOL}\end{aligned}$$

2.11 Function Call

$$\text{FunctionCall} \rightarrow \text{ID}(\text{ArgList})$$

$$\begin{aligned}\text{ArgList} \rightarrow\ & \text{Expr}\ ,\ \text{ArgList} \\ |\ & \text{Expr} \\ |\ & \varepsilon\end{aligned}$$

3 Regular Expression for Tokens

3.1 Keywords

Token	Regular Expression	Description
CLASS	`class`	Start of class definition
ELSE	`else`	Alternative branch of if
FI	`fi`	End of if statement
IF	`if`	Conditional start
IN	`in`	Let expression body delimiter
LET	`let`	Variable declaration start
LOOP	`loop`	Start of while loop body
POOL	`pool`	End of while loop body
THEN	`then`	Consequent branch of if
WHILE	`while`	While loop start
NOT	`not`	Logical negation
AND	`and`	Logical conjunction

3.2 Operators and Delimiters

Token	Regular Expression	Description
ASSIGN	`<-`	Assignment operator
LPAREN	`(`	Left parenthesis
RPAREN	`)`	Right parenthesis
LBRACE	`{`	Left curly brace
RBRACE	`}`	Right curly brace
COLON	`:`	Type declaration separator
SEMICOLON	`;`	Statement terminator
COMMA	`,`	List separator
PLUS	`+`	Addition
MINUS	`-`	Subtraction
MULT	`*`	Multiplication
DIV	`/`	Division
LE	`<=`	Less than or equal comparison
LT	`<`	Less than comparison

3.3 Identifiers and Types

Token	Regular Expression	Description
TYPE	`[A-Z][a-zA-Z0-9_]*`	Must start with uppercase (e.g., Int, Main)
ID	`[a-z][a-zA-Z0-9_]*`	Must start with lowercase (e.g., x, main)

3.4 Literals

Token	Regular Expression	Description
INT	`[0-9]+`	Sequence of digits
BOOL	`true` | `false`	Boolean constants
STRING	`"([^"\n ]|\\.)*"`	Characters enclosed in double quotes

3.5 Whitespace and Comments

Category	Regular Expression	Action
WS	`[ \t \r \n ]+`	Skip whitespace
COMMENT	`-.*`	Skip single-line comments
B-COMMENT	`\(\*([^*]`[)]`)*|`	Skip block comments

4 Setup Instructions

Execution Steps

1. **Extract the Files:** Ensure the following files are located in your current working directory:

   - `cool.l`
   - `validprogram.cl`
   - `invalidprogram.cl`

2. **Compile the Lex File:** Generate the C source code from the Lex specification by executing the following command:

   ```
   lex cool.l
   ```

3. **Compile the C Code:** Compile the newly generated `lex.yy.c` file into an executable. You must link the Flex library using the `-ll` flag:

   ```
   gcc lex.yy.c -o lexer -ll
   ```

4. **Run the Lexer:** The lexer is designed to process `validprogram.cl` by default. If you want to process a different file, provide it as a command-line argument. In both cases, it redirects the output to `output.txt`.

   To run with the default file (`validprogram.cl`):

   ```
   ./lexer
   ```

   To run with a custom file (e.g., `invalidprogram.cl`):

   ```
   ./lexer invalidprogram.cl
   ```

5 Lexical Errors

The following errors are in the `invalidprogram.cl`

- Line 08: There is no `=` operator in the language.
- Line 09: Identifier cannot start with a number
- Line 21: There is no `>` operator in the language. any conditional should only use $<$ or `<=`
- Line 36: Multiline Comment bracket doesnt close and goes to EOF